TOETAE SPECIAL

Schema overview and API

1 Schema

`permission`(<u>id</u>, can_overwrite, can_edit_role)

`role`(<u>id</u>, name, description, *permission_id*†)

`account`(<u>id</u>, username, email, *role_id*†, quota, active)

`status`(<u>id</u>, name)

`category`(<u>id</u>, name)

`task`(<u>id</u>, name, description, *status_id*†, *category_id*†, *created_by*†, *completed_by*†, created, updated, completed_at, due_date)

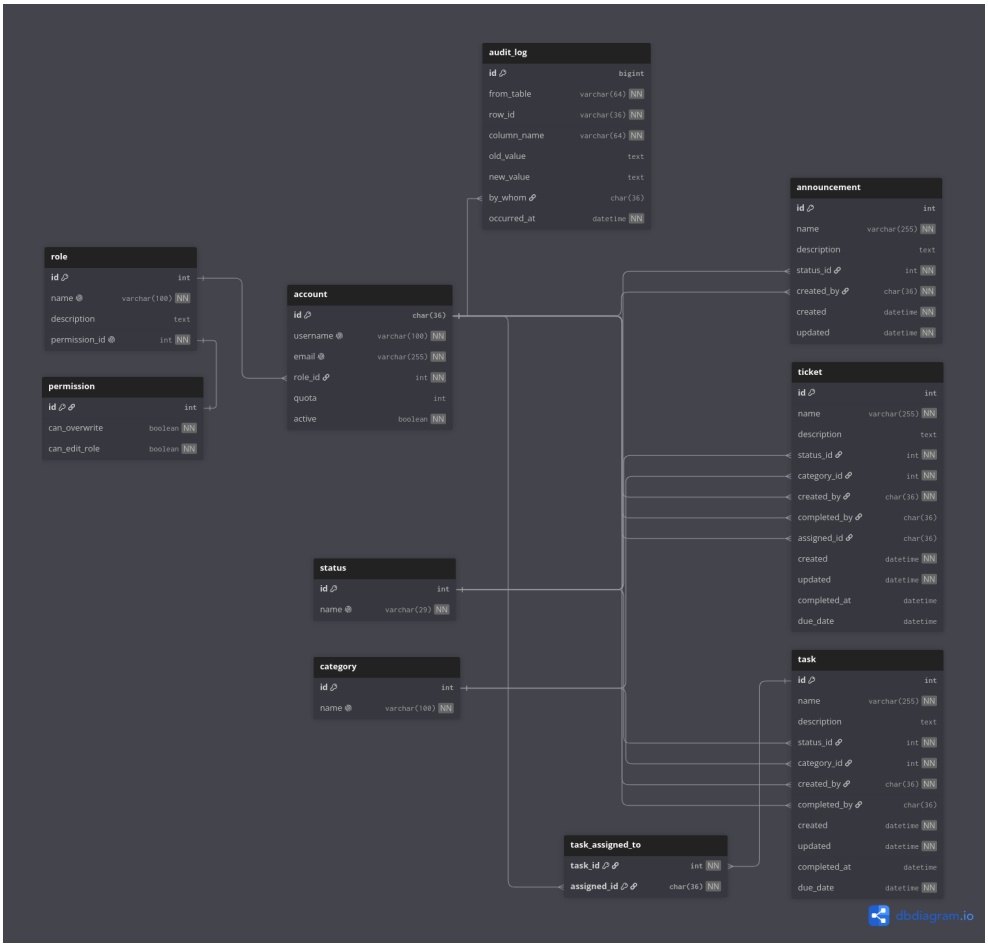
`task_assigned_to`(<u>*task_id*†, *assigned_id*†</u>)

`ticket`(<u>id</u>, name, description, *status_id*†, *category_id*†, *created_by*†, *completed_by*†, *assigned_id*†, created, updated, completed_at, due_date)

`announcement`(<u>id</u>, name, description, *status_id*†, *created_by*†, created, updated)

`audit_log`(<u>id</u>, from_table, row_id, column_name, old_value, new_value, *by_whom*†, occurred_at)

<u>Underlined</u> = primary key. *Italic*† = foreign key.



2 Before We Begin

Before implementation begins, please read through the API requirements below.

The purpose of this section is to establish a few common agreements so that the backend and frontend can be implemented without making conflicting assumptions.

2.1 Roles

Role	Permissions
Lab Admins	Have access to create, edit, update, and delete every request (tickets and tasks). They may also create user accounts and grant roles. User accounts are never deleted. Instead, an account is deactivated by setting `active` to `false`.
Lab Users	May create, edit, update, and delete only their own requests. The exception is a task assigned to everyone. In that case, any user may update the task's status.

2.2 Status

The `status` table is fixed.

The workflow always uses the same five status values, and records reference them through `status_id`.

The `status` query parameter described later accepts only these five status names.

ID	Name	Description
1	Pending	Newly submitted and not yet reviewed.
2	Approve	Reviewed and approved. Work has not necessarily started yet.
3	Reject	Reviewed and declined. No further action is expected.
4	In-Progress	Approved and currently being worked on.
5	Complete	Finished. Excluded from the dashboard view and visible only in the request table.

Status ordering can simply follow `status.id`, which naturally produces:

`Pending` → `Approve` → `Reject` → `In-Progress` → `Complete`

2.3 Category

Unlike `status`, the `category` table is dynamic.

Lab staff may define their own categories. The following rows are example seed data only and are **not** part of the fixed specification.

ID	Name	Description
1	Safety & Compliance	Example only.
2	Reagents & Consumables	Example only.
3	Sample Prep	Example only.
4	Instrumentation	Example only.
5	Data Analysis	Example only.

Because categories are configurable, no endpoint should hard-code a category name or assume that a particular `category_id` will always exist.

3 API Requirements

This is a guide, not a prison.

The response structures shown below are suggested examples to make frontend development easier. The final implementation does not have to follow them character-for-character. As long as the API remains consistent, predictable, and documented, feel free to structure the endpoints however you think is best.

4 GET /tasks

Retrieves tasks available to the current user.

4.1 Optional Query Parameters

Parameter	Description
`status`	Filter tasks by status. Only the five predefined status names are accepted.
`category`	Filter tasks by category name.
`onlyDueDate`	When `true`, return only tasks whose due date has not passed.

4.2 Dashboard Behaviour

When `onlyDueDate=true` is supplied without any other filters, the endpoint is intended to provide the tasks shown on the dashboard.

For the dashboard, only tasks with status `In-Progress` should be returned.

Caller	Behaviour
Admin	Returns every task. No ownership or assignment filter is applied. Results are ordered by `status.id`.
Non-admin	Returns only tasks assigned to the caller through `task_assigned_to`. Results are ordered by due date, followed by status.

The scope of a non-admin user's task list is determined by the assignment table, **not** by `created_by`.

Therefore, a user who created a task but was never assigned to it does not automatically see the task on their dashboard.

4.3 Example Response


```
{
  "Tasks": [
    {
      "name": "Validate autoclave cycle with spore strips",
      "description": "...",
      "status": "In-Progress",
      "category": "Safety & Compliance",
      "due_date": "2026-08-20T17:00:00",
      "created_by": "h.nakamura"
    },
    {
      "name": "Reconcile reagent inventory against orders",
      "description": "...",
      "status": "In-Progress",
      "category": "Reagents & Consumables",
      "due_date": "2026-09-02T17:00:00",
      "created_by": "h.nakamura"
    },
  ]
}
```


No task in the seed data is `Pending`, `Approve`, or `Reject`.

Therefore, the admin list contains six `In-Progress` tasks followed by two `Complete` tasks.

4.4 Example: Non-admin User

For user `h.nakamura`, four tasks are assigned to the


```
{
  "Tasks": [
    {
      "name": "Validate autoclave cycle with spore strips",
      "description": "...",
      "status": "In-Progress",
      "category": "Safety & Compliance",
      "due_date": "2026-08-20T17:00:00",
      "created_by": "h.nakamura"
    },
    {
      "name": "Re-run failed qPCR plate 14",
      "description": "...",
      "status": "In-Progress",
      "category": "Sample Prep",
      "due_date": "2026-09-19T17:00:00",
      "created_by": "h.nakamura"
    },
  ]
}
```

5 GET /tickets

Retrieves tickets available to the current user.

5.1 Optional Query Parameters

Parameter	Description
`status`	Filter tickets by status.
`category`	Filter tickets by category.
`onlyOwned`	When `true`, return only tickets created by the caller.

5.2 Access Behaviour

Caller	Behaviour
Admin	Returns every ticket. No ownership filter is applied. Results are ordered by `status.id`.
Non-admin	Returns only tickets created by the caller: `ticket.created_by = :me` Results are ordered by due date, followed by status.

5.3 Example: Admin

For example, an admin user such as `e.vasquez` receives all seven tickets.


```
{
  "Tickets": [
    {
      "name": "Wrong lot of antibody shipped",
      "description": "...",
      "status": "Pending",
      "category": "Reagents & Consumables",
      "due_date": "2026-09-11T17:00:00",
      "created_by": "h.nakamura"
    },
    {
      "name": "Centrifuge vibrating at high speed",
      "description": "...",
      "status": "Pending",
      "category": "Instrumentation",
      "due_date": null,
      "created_by": "d.okonkwo"
    }
  ]
}
```


All five statuses appear in `status.id` order:

Pending → Approve → Reject → In-Progress → Complete

5.4 Example: Non-admin User

For user `h.nakamura`, three tickets were created by the user.


```
{
  "Tickets": [
    {
      "name": "Freezer temperature log has gaps",
      "description": "...",
      "status": "Complete",
      "category": "Safety & Compliance",
      "due_date": "2026-07-29T17:00:00",
      "created_by": "h.nakamura"
    },
    {
      "name": "Request BSL-2 room access for new student",
      "description": "...",
      "status": "In-Progress",
      "category": "Safety & Compliance",
      "due_date": "2026-08-22T17:00:00",
      "created_by": "h.nakamura"
    },
    {
      "name": "Wrong lot of antibody shipped",
      "description": "...",
      "status": "Pending",
      "category": "Reagents & Consumables",
      "due_date": "2026-09-11T17:00:00",
      "created_by": "h.nakamura"
    }
  ]
}
```


Ticket 1 and Ticket 3 are both assigned to `h.nakamura`, but they were created by `d.okonkwo`.

Therefore, they do not appear in the non-admin result when using the ownership rule above.

If the dashboard is intended to show all work a technician needs to act on, the predicate can instead be:

```
ticket.created_by = :me
OR ticket.assigned_id = :me
```

6 POST / PUT / DELETE

The exact implementation of the mutation endpoints is intentionally left open.

Use whatever request and response structure makes the most sense for the backend implementation, provided that the following requirements are respected:

- Authentication and authorization must be enforced.
- Admin users must be able to perform their permitted operations across all requests.
- Non-admin users must not modify requests they do not have permission to modify.
- User accounts must never be physically deleted. Deactivate them using `active=false`.
- Changes to important records should be compatible with the `audit_log` table.
- Status and category references must remain valid.
- The API should return appropriate HTTP status codes.
- Request and response formats should remain consistent across endpoints.

POST? PUT? DELETE?

Do whatever makes the most sense. Just don't break the database, authorization rules, or whoever has to consume the API afterwards.

7 General API Principles

The following principles apply to all endpoints:

1. The authenticated user determines the caller's permissions.
2. Authorization must be enforced by the backend. The frontend must not be treated as a security boundary.
3. Admin permissions are determined by the user's role and associated permissions.
4. Non-admin access is restricted according to the ownership and assignment rules described above.
5. Statuses are fixed and must not be dynamically created by normal request operations.
6. Categories are dynamic and must be resolved through the `category` table.
7. Dates should use a consistent ISO 8601-compatible representation on the database. For frontend just convert it to Thailand timezone.
8. Deleted or deactivated resources should behave consistently across all endpoints.
9. API behaviour should be documented whenever the implementation differs from the examples in this document.

8 Final Note

The examples in this document are intended to communicate **behaviour**, not to dictate every implementation detail.

If the backend implementation requires a different response shape, additional fields, different mutation routes, or other improvements, that is completely fine.

Build it however you want.

Just don't leak the API keys